Implementation

Engine Overview

TIVIDY has two halves: the definition library and the running instances. A WorkflowDefinition holds a template tree of stages and tasks, a set of routing and assignment policies, and an escalation chain. It is never executed. Each time a user starts a workflow, WorkflowDefinition::createInstance() deep-clones the template tree into a WorkflowInstance, which owns the clone plus its own WorkflowData, EventPublisher, AuditLog, and ActionHistory.

The Tividy façade sits above both halves. The TUI talks only to Tividy. It never touches a WorkflowInstance, WorkItem, or Command directly. Tividy holds the installed packs, the workflow library, all running instances, the participant directory, the connector registry, and a ManualClock.

Starting a Workflow Instance

Tividy::startWorkflow(name, data) does four things. It looks up the definition by name. It allocates a fresh instance id and calls createInstance(id, data, services), which clones the template tree and binds the clone to a fresh WorkContext. It attaches a NotificationObserver to the instance's EventPublisher (the AuditObserver and FlowController were attached in the WorkflowInstance constructor). It calls instance.start().

start() transitions the instance from Created to Running and asks FlowController to release every eligible item. An item is eligible if its status is Created and every item listed in its WorkItemSpec::dependencies() has settled (Completed or Skipped). FlowController walks the tree with a DepthFirstIterator, publishes every eligible item, and each ItemAvailable event triggers an assignment attempt through the configured AssignmentStrategy. If the strategy returns a participant id, the item transitions to Assigned. If the item is marked automatic, FlowController immediately starts and completes it, which can in turn release further items.

Taking a User Action

The TUI calls Tividy::perform(verb, instanceId, itemId, actorId, outcome, text). Inside Tividy, this resolves to CommandFactory::create(verb, request) followed by ActionHistory::execute.

InstanceCommand::execute follows a fixed sequence. It finds the item by id. It captures an InstanceMemento of the whole instance, used later for undo. It calls the subclass's doExecute, which delegates to the item's lifecycle method. That call goes to the current WorkItemState. For example, CompleteCommand::doExecute calls item.complete(actor, outcome), which calls state.complete(*this, actor, outcome), and InProgressState::complete is the only state that accepts this call.

The state class runs the Template Method: validateOutcome, performWork, onCompleted. performWork is the industry-specific operation. In banking it might post a ledger entry through an ExternalConnector. In telecom it might provision a network element. On success, the state transitions the item to Completed and publishes an ItemCompleted event. On failure, the template publishes ItemFailed and returns without transitioning.

How the Workflow Progresses

FlowController is the progression engine, implemented as an observer on the instance's EventPublisher. Every event that affects an item reaches FlowController after it has been written to the audit log.

On ItemCompleted, FlowController looks up the RoutingStrategy for the completed item's id from DefinitionPolicies::routing. It calls the strategy's decide method, which returns a list of items to activate and a list to skip. FlowController publishes the activated items and skips the rest. It then walks the tree to release any items whose dependencies are now satisfied.

On ItemFailed, FlowController compares the failure count against WorkItemSpec::maxRetries(). If retries remain, it re-runs the item (for automatic items) or leaves it available for the user to retry (for manual items). If retries are exhausted, it escalates the item through the configured EscalationHandler chain.

On ItemEscalated, FlowController walks the escalation chain. Each RoleEscalationHandler checks whether the escalation level matches its configured level. The first match returns an approver role, and FlowController assigns the escalated item to a participant holding that role.

All of these reactions run inside FlowController::onEvent, but not recursively. The controller pushes every incoming event onto a pending queue and drains it iteratively. A reaction that publishes more events enqueues them at the back of the queue rather than calling back into onEvent.

How Deadlines Are Handled

Tividy runs on simulated time via ManualClock. Tividy::advanceTime(seconds) advances the clock and sweeps every Running instance via its DeadlineMonitor. The monitor walks the tree, finds any item whose dueAt() has passed and whose status is not already terminal, and publishes DeadlineMissed.

FlowController reacts to DeadlineMissed based on WorkItemSpec::timeoutCode(). If the code is set, the item expires with that outcome (a system completion that routing can branch on, such as "merchant silent for 14 days, default judgment for customer"). If the code is empty, the item escalates through the standard chain.

How Industries Plug In

An industry is a subclass of IndustryPack. The pack declares three pure virtual methods (key, displayName, createDirector) and can override five others for participants,

connectors, assignment strategy, escalation chain, and UI profile. The defaults are correct for the simplest case: no participants, a LoggingNotifier, role-based assignment, no escalation chain, and a neutral profile.

A Director subclass declares the workflow. Its construct(builder) method issues builder calls (beginStage, addTask, setRouting, setEscalationChain) describing the stages, tasks, dependencies, and policies. The Director uses StandardWorkflowBuilder, which enforces structural invariants as the definition is assembled and validates the result before returning.

Tividy::installPack(pack) does five things. It stores the pack. It calls pack.createParticipants() and registers each participant. It calls pack.createConnectors() and registers each connector. It calls pack.createDirector().build(StandardWorkflowBuilder) to construct the definition, then adds it to the library keyed by pack key. It stores pack.createProfile() for later use by the TUI.

After installation, Tividy.workflowNames() returns the newly registered workflow and Tividy.startWorkflow can be called with it. The industry is not mentioned anywhere in the engine. Adding a third industry is one IndustryPack subclass, one WorkflowDirector subclass, and whatever tasks and connectors the workflow needs.